

Final Exam Review

CSE3031 Computer Architecture

Dr. Jun Zheng

5/5/2026



Final Exam

- Wednesday, May 13, 1 pm - 3:30 pm
- Cramer 203
- Close book, close notes, one side sheet (A4, both sides) and calculator are allowed.

What's Covered?

- 20%
 - Fundamentals of Computer Design
 - Instruction set principles and examples
 - Memory hierarchy
- 80%
 - Memory hierarchy - virtual memory
 - Advanced memory hierarchy
 - Pipelining basics
 - Instructional-level parallelism

Sample Question 1

- Victim cache can reduce conflict misses.
 - True
- Pipelining can reduce the latency of a task.
 - False
- Scoreboard resolves RAW hazard at the issue stage
 - False

Sample Question 2 - HW 3

Q1

- Consider a 64-byte cache with 8 byte blocks, an associativity of 2 and LRU block replacement. Virtual addresses are 16 bits. The cache is physically tagged. The processor has 16KB of physical memory (Hint: Calculate the physical address size based on the physical memory size).
- a. What is the total number of tag bits?
- b. If the cache the virtually indexed, what is the minimum page size to avoid the synonym problem?
- c. Assume each page is 64 bytes. How large would a page table be given that each page requires 4 protection bits in addition to the physical page number, and entries must be an integral number of bytes?
- d. Why are the first-level caches usually split (instructions and data are in different caches) while the L2 cache is usually unified (instructions and data are both in the same cache)?

Answer

A.

The cache is 64-bytes with 8-byte blocks, so there are 8 blocks. The associativity is 2, so there are 4 sets. Since there are 16KB of physical memory, a physical address is 14 bits long. Of these, 3 bits are taken for the offset (8-byte blocks), and 2 for the index (4 sets). That leaves 9 tag bits per block. Since there are 8 blocks, that makes 72 tag bits or 9 tag bytes.

B.

To avoid synonyms, the number of sets times the block size cannot exceed the page size. Hence, the minimum page size is 32 bytes

C.

There is 16KB of physical memory and 64 B pages, meaning there are 256 physical pages and each PFN is 8 bits. Each entry has a PFN and 4 protection bits, that's 12 bits which we round up to 2 bytes. A single-level page table has one entry per virtual page. With a 16-bit virtual address space, there is 64KB of memory. Since each page is 64 bytes, that means there are 1K pages. At 2 bytes per page, the page table is 2KB in size.

D.

The primary reason for splitting the first level cache is bandwidth, i.e., to avoid structural hazards. A typical processor must access instructions and data in the same cycle quite frequently. Supplying this kind of bandwidth in a unified cache requires either replication (in which case you may as well have a split cache) or multi-porting or multi-banking, both of which would slow down the hit time. Structural hazards are a much smaller issue for a second-level cache which is accessed much less frequently. Here the primary design goal of a second level cache is a low miss-rate. By combining instructions and data in the same cache, the capacity of the cache will be better utilized and a lower overall miss rate may be achieved.

Sample Question 3 - HW3

Q2

- Consider the code snippet below. Suppose it is executed on a system with a 2-way set associative 16KB data cache with 32-byte blocks, 32-bit words, and an LRU replacement policy. Assume that `int` is word-sized. Also assume that the address of `a` is `0x0`, that `i` and `x` are in registers, and that the cache is initially empty. How many data cache misses are there? How many hits are there? Please justify your answer.

```
int i;  
int a[1024*1024];  
int x = 0;  
for (i = 0; i < 1024; i++)  
{  
    x += a[i] + a[1024*i];  
}
```

Solution

- The number of sets in the cache = $16\text{KB}/(2*32\text{B}) = 256$
- Since a word is 4B, int is word sized and block size is 32B, a block can hold 8 ints.
- $a[0]$ to $a[1023]$ will be mapped to one of cache lines in sets 0 to 127. Every 8 elements will generate 1 miss and 7 hits. In total, 128 misses and 896 hits
- For $a[0*1024]$, $a[1*1024]$, ..., $a[1023*1024]$, except the access to $a[0]$ is a hit, all other accesses will be misses as each of them will be mapped to a new cache line. In total, 1023 misses and 1 hit.
- Total: 1151 misses and 897 hits

Sample Question 4 - HW 4

Q2

- The instruction execution of a computer can be divided into five stages as follows: IF, ID, EX, MEM and WB. The measured times for the stages are: 1ns, 1.5ns, 1ns, 2ns, and 1.5ns, respectively.
 - (a) What is the clock cycle time if the instruction execution is implemented as a single-cycle implementation?
 - (b) What is the clock cycle time if the instruction execution is implemented as a multiple-cycle implementation?
 - (c) What is the clock cycle time if the instruction execution is implemented as a pipelined implementation?
 - (d) What is the speedup of the pipelined implementation over the single-cycle implementation?

Solution

- (a) 7ns
- (b) 2ns
- (c) 2ns
- (d) Both CPI = 1, IC the same, speedup is based on cycle time, $7\text{ns}/2\text{ns} = 3.5$

Sample Question 5 - HW4 Q3

- For a deeper pipeline, such as that in MIPS R4000, it takes three pipeline stage before the branch target address is known and an additional cycle before the branch condition is evaluated (taken or not taken) (i.e. the branch target address is calculated at stage 3 and branch result is evaluated at stage 4). Assume no stalls on the registers in the conditional comparison.
- Fill in the table for the number of cycles of penalty for the three static branch approaches. Note: if what you have fetched is correct, you do not need to repeat the instruction fetch.

Branch Scheme	Penalty unconditional	Penalty conditional taken	Penalty conditional untaken
Stall pipeline until branch result detected			
Predicted taken			
Predicted untaken			

Solution

Branch Scheme	Penalty unconditional	Penalty taken	Penalty untaken
Stall pipeline until branch result detected	2	3	3
Predicted taken	2	2	3
Predicted untaken	2	3	0

Sample Question 6 - HW 4

Q5

- Consider a loop that is entered two times in sequence in a program. Each time it is entered, the loop performs 4 iterations. Each iteration executes two branches in sequence with the following outcomes:

Iteration	1	2	3	4	1	2	3	4
Branch 1	T	N	T	N	T	N	T	N
Branch 2	T	T	T	N	T	T	T	N

- When Branch 2 is not taken at iteration 4, the program leaves the loop. Record the predictions of the two branches using the following two branch predictors: (1) a 2-bit dynamic up/down saturation counter predictor (initial state 00), (2) a (1,1) correlated predictor (initial state N/N). Assume that the outcome of the branch instruction executed before entering the loop is N.

2-bit Saturation Counter Predictor

Branch 1

Iteration	1	2	3	4	1	2	3	4
Branch Outcome	T	N	T	N	T	N	T	N
Predictor State	00							
Prediction								

Branch 2

Iteration	1	2	3	4	1	2	3	4
Branch Outcome	T	T	T	N	T	T	T	N
Predictor State	00							
Prediction								

Answer

b1:

Iteration	1	2	3	4	1	2	3	4
Branch outcome	T	N	T	N	T	N	T	N
Predictor state	00	01	00	01	00	01	00	01
Prediction	N	N	N	N	N	N	N	N

b2:

Iteration	1	2	3	4	1	2	3	4
Branch outcome	T	T	T	N	T	T	T	N
Predictor state	00	01	10	11	10	11	11	11
Prediction	N	N	T	T	T	T	T	T

(1, 1) Correlated Predictor

Branch 1

Iteration	1	2	3	4	1	2	3	4
Branch Outcome	T	N	T	N	T	N	T	N
Predictor State	N/N							
Prediction								

Branch 2

Iteration	1	2	3	4	1	2	3	4
Branch Outcome	T	T	T	N	T	T	T	N
Predictor State	N/N							
Prediction								

Answer

b1:

Iteration	1	2	3	4	1	2	3	4
Previous branch outcome	N	T (b2)	T(b2)	T(b2)	N(b2)	T(b2)	T(b2)	T(b2)
Branch outcome	T	N	T	N	T	N	T	N
Predictor state	N/N	T/N	T/N	T/T	T/N	T/N	T/N	T/T
Prediction	N	N	N	T	T	N	N	T

b2:

Iteration	1	2	3	4	1	2	3	4
Previous branch outcome	T (b1)	N(b1)	T(b1)	N(b1)	T(b1)	N(b1)	T(b1)	N(b1)
Branch outcome	T	T	T	N	T	T	T	N
Predictor state	N/N	N/T	T/T	T/T	N/T	N/T	T/T	T/T
Prediction	N	N	T	T	T	N	T	T

Sample Question 7

(1) Considering the following code fragment, list all dependencies (true, anti, output).

1: $F_4 \leftarrow F_2 - F_{12}$

2: $F_0 \leftarrow F_2 * F_4$

3: $F_6 \leftarrow F_2 / F_8$

4: $F_8 \leftarrow F_2 + F_{12}$

5: $F_6 \leftarrow F_{14} + F_{16}$

Answer

Instruction 1	Instruction 2	Register	Dependency (true/anti/output)
i1	i2	F4	True
i3	i4	F8	Anti
i3	i5	F6	output

(2)

- For the code fragment given in (1), complete the following table using a CDC6600-like processor with scoreboard. Assume **two FP multipliers (FP Mult)**, **one FP adder and one FP divider**. FP multipliers take 7 cycles, FP adder takes 4 cycles and FP divider takes 25 cycles.

Instruction	Functional Unit	Issue	Dispatch	Execution	Write Result
$F4 \leftarrow F2 - F12$	FP Adder	1	2	3-6	7
$F0 \leftarrow F2 * F4$	FP Mult 1	2	8	9-15	16
$F6 \leftarrow F2 / F8$	FP Divider	3	4	5-29	30
$F8 \leftarrow F2 + F12$	FP Adder	8	9	10-13	14
$F6 \leftarrow F14 + F16$	FP Adder	31	32	33-36	37

(3)

- For the code fragment given in (1), complete the following table using an IBM360-like processor with Tomasulo's algorithm (Functional units are not pipelined and no forwarding). Assume **one FP adder with 3 reserve stations (Add1 – Add3)** and **one FP multiplier/divider with 2 reserve stations (Mult1 and Mult2)**. FP multiplication takes 7 cycles, FP addition and subtraction take 4 cycles and FP division takes 25 cycles.

Instruction	Reserve station	Issue	Execution	Write Result
$F4 \leftarrow F2 - F12$	Add1	1	2-5	6
$F0 \leftarrow F2 * F4$	Mult1	2	29-35	36
$F6 \leftarrow F2/F8$	Mult2	3	4-28	29
$F8 \leftarrow F2 + F12$	Add2	4	6-9	10
$F6 \leftarrow F14 + F16$	Add3	5	10-13	14